

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 4

Student Name: V. Lekhya sri

Roll No.: A24126552123

Date: 25-08-2026

1. TITLE

Develop Dynamic Webpage Using Java Script

2. AIM

- To develop a dynamic webpage using **HTML, CSS, and JavaScript**.
 - To create a **To-Do List application** that allows users to add, complete, and delete tasks dynamically.
-

3. OBJECTIVE

The objectives of this experiment are:

- To understand the concept of developing dynamic webpages using JavaScript.
 - To use JavaScript functions to perform different operations.
 - To dynamically create and add HTML elements using JavaScript.
 - To implement functionality for adding tasks to a To-Do List.
 - To mark completed tasks using JavaScript and CSS classes.
 - To delete tasks dynamically from the webpage.
 - To use DOM methods to access, create, modify, and remove HTML elements.
-

4. THEORY

JavaScript is a programming language used to make webpages dynamic and interactive. It allows webpage content to be created, modified, and removed based on user actions.

In this application, JavaScript is used to develop a dynamic **To-Do List** that allows users to manage their daily tasks.

- **HTML** is used to create the structure of the To-Do List.
- **CSS** is used to style the webpage, tasks, and buttons.
- **JavaScript functions** are used to perform task-related operations.
- **DOM methods** are used to access and modify HTML elements dynamically.
- `document.createElement()` is used to create new task elements.
- `appendChild()` is used to add tasks and buttons to the webpage.
- `classList.toggle()` is used to mark a task as completed or uncompleted.
- `remove()` is used to delete a task from the list.
- **Event handling** is used to perform actions when buttons are clicked.

The basic flow of the application is:

User Input → Add Task Function → Dynamic HTML Element Creation → Display Task → Complete/Delete Action → Updated To-Do List

5. ALGORITHM / PROCEDURE

1. Create a new HTML file.
 2. Add the basic HTML document structure using `<!DOCTYPE html>`, `<html>`, `<head>`, and `<body>`.
 3. Add CSS styles to design the To-Do List webpage.
 4. Create a container to hold the To-Do List application.
 5. Add an input field to accept tasks from the user.
 6. Add an **Add** button to call the `addTask()` function.
 7. Create an empty unordered list to display the tasks.
 8. Use `document.getElementById()` to access the task input field.
 9. Check whether the entered task is empty.
 10. Display an alert message if no task is entered.
 11. Call the `createTask()` function when a valid task is entered.
 12. Use `document.createElement()` to dynamically create a list item, task text, Complete button, and Delete button.
 13. Use `appendChild()` to add the task and buttons to the list item.
 14. Add the newly created task to the To-Do List.
 15. Use `classList.toggle()` to mark a task as completed or uncompleted.
 16. Use `remove()` to delete a task from the list.
 17. Clear the input field after adding a task.
 18. Open the HTML file in a web browser.
 19. Add, complete, and delete different tasks to verify the dynamic functionality.
-

6. PROGRAM

```
<!DOCTYPE html>
<html>

<head>
  <title>To-Do List</title>

  <style>
    body {
      font-family: Arial, sans-serif;
      background: #2bbdc3;
      text-align: center;
      padding-top: 60px;
    }

    .container {
      background: white;
      width: 450px;
      margin: auto;
      padding: 30px;
      border-radius: 12px;
      box-shadow: 0 4px 15px rgba(0, 0, 0, 0.1);
    }

    input {
      padding: 12px;
      width: 65%;
      border: 1px solid #ccc;
      border-radius: 6px;
    }

    .add {
      padding: 12px 18px;
      border: none;
```

```
        border-radius: 6px;
        background: #333;
        color: white;
        cursor: pointer;
    }

    ul {
        padding: 0;
        text-align: left;
    }

    li {
        list-style: none;
        padding: 12px;
        margin-top: 10px;
        background: #f8f8f8;
        border-radius: 6px;
        font-style: italic;
    }

    li button {
        float: right;
        margin-left: 6px;
        border: none;
        padding: 6px 9px;
        border-radius: 4px;
        cursor: pointer;
    }

    .complete {
        background: #e0e0e0;
    }

    .delete {
        background: #ffdddd;
    }

    .completed {
```

```
        text-decoration: line-through;
        color: #888;
    }
</style>
</head>

<body>

    <div class="container">

        <h1>My To-Do List</h1>

        <input id="taskInput" type="text" placeholder="What's the plan
for today?">

        <button class="add" onclick="addTask()">Add</button>

        <ul id="taskList"></ul>

    </div>

    <script>

        function addTask() {

            let input = document.getElementById("taskInput");
            let taskText = input.value;

            if (taskText === "") {
                alert("Please enter a task!");
                return;
            }

            createTask(taskText);

            input.value = "";
        }
    </script>
</body>
</html>
```

```
function createTask(taskText) {

    let li = document.createElement("li");

    let task = document.createElement("span");
    task.textContent = taskText;

    // Complete button
    let completeButton = document.createElement("button");

    completeButton.textContent = "done";
    completeButton.className = "complete";

    completeButton.onclick = () => {
        completeTask(task);
    };

    // Delete button
    let deleteButton = document.createElement("button");

    deleteButton.textContent = "delete";
    deleteButton.className = "delete";

    deleteButton.onclick = () => {
        deleteTask(li);
    };

    li.appendChild(task);
    li.appendChild(deleteButton);
    li.appendChild(completeButton);

    document.getElementById("taskList").appendChild(li);
}
```

```

        function completeTask(task) {

            task.classList.toggle("completed");
        }

        function deleteTask(task) {

            task.remove();
        }

    </script>

</body>

</html>

```

7. CODE EXPLANATION

JavaScript / HTML Component	Description
<!DOCTYPE html>	Defines the document as an HTML5 webpage.
<div class="container">	Creates the main container for the To-Do List application.
<input id="taskInput">	Accepts a task entered by the user.
onclick="addTask()"	Calls the <code>addTask()</code> function when the Add button is clicked.
<ul id="taskList">	Creates an unordered list to display the tasks.
function addTask()	Retrieves the user input and adds a valid task to the list.
document.getElementById()	Accesses an HTML element using its ID.

<code>input.value</code>	Retrieves or modifies the value entered in the input field.
<code>if (taskText === "")</code>	Checks whether the user entered an empty task.
<code>alert()</code>	Displays an alert message when no task is entered.
<code>function createTask()</code>	Dynamically creates a new task and its buttons.
<code>document.createElement()</code>	Creates new HTML elements dynamically using JavaScript.
<code>textContent</code>	Sets the text content of an HTML element.
<code>appendChild()</code>	Adds an element as a child of another HTML element.
<code>completeButton</code>	Creates a button used to mark a task as completed.
<code>deleteButton</code>	Creates a button used to delete a task.
<code>onclick()</code>	Defines the action performed when a button is clicked.
<code>function completeTask()</code>	Marks or removes the completed status of a task.
<code>classList.toggle("completed")</code>	Adds or removes the completed CSS class from a task.
<code>.completed</code>	Applies a line-through style to completed tasks.
<code>function deleteTask()</code>	Defines the function used to delete a task.
<code>remove()</code>	Removes the selected task element from the webpage.
CSS Selectors	Select and style the HTML elements and classes.
CSS border-radius	Creates rounded corners for the container, input fields, and buttons.
CSS box-shadow	Adds a shadow effect to the main container.

8. TEST CASES AND EXECUTION DATA

Test Case	Test Scenario	Input / Action	Expected Result	Actual Result	Status
TC-01	Open webpage	Open HTML file in browser	To-Do List webpage should load correctly	Webpage loaded correctly	Pass
TC-02	Add a task	Enter Complete Assignment and click Add	Task should be added to the list	Task added successfully	Pass
TC-03	Add multiple tasks	Enter multiple tasks and click Add	All tasks should be displayed in the list	Tasks displayed correctly	Pass
TC-04	Add empty task	Leave the input field empty and click Add	Alert message should be displayed	Alert displayed correctly	Pass
TC-05	Complete a task	Click the done button	Task should be marked with a line-through	Task marked as completed	Pass
TC-06	Uncomplete a task	Click the done button again	Completed style should be removed	Completed style removed correctly	Pass
TC-07	Delete a task	Click the delete button	Click the delete button	Task deleted successfully	Pass
TC-08	Check dynamic task creation	Add a new task	New element and buttons should be created dynamically	Elements created correctly	Pass

9. OUTPUT

My To-Do List

What's the plan for today?

Add

~~do some research~~

done

delete

My To-Do List

What's the plan for today?

Add

do some research

done

delete

My To-Do List

What's the plan for today?

Add

Please enter a task!

Close

My To-Do List

What's the plan for today?

Add

10. RESULT

Thus, a **dynamic To-Do List webpage** was successfully developed using **HTML, CSS, and JavaScript**. The application allows users to dynamically add tasks, mark tasks as completed, and delete tasks. JavaScript DOM manipulation and functions were successfully implemented, and the application was verified using multiple test cases.